VI - TP 1 Dictionnaire, associations clé-valeur

1 On utilise un dictionnaire pour associer à un nom d'utilisateur son mot de passe.

Code python
```python
mdp = {"Alice": "MVTMJSUNP", "Bob": "0123"}
```

1. Que renvoie `mdp["Alice"]` ? Quelle instruction renvoie le mot de passe de Bob ?
2. Que renvoie `mpd["Cléo"]` ? `mdp["0123"]` ? Commenter.
3. Exécuter `mpd["Charles"] = "azerty"`, puis afficher le dictionnaire `mdp`. Commenter.
4. Ajouter au dictionnaire l'utilisateur Évelyne, dont le mot de passe est `"fustell"`.
5. Bob souhaite modifier son mot de passe en `"Monsup3rmotdep@sse!!!"`.
   Écrire l'instruction python correspondante.
6. **a.** Exécuter les instructions :

   i. `"Alice" in mdp` **iii.** `"Cléo" in mdp` **v.** `not ("Bob" in mdp)`
   ii. `"Bob" in mdp` **iv.** `"0123" in mdp` **vi.** `"Cléo" not in mpd`

   b. De manière générale, que renvoie `k in mdp` ?
7. Écrire une fonction `verifie`, qui prend en argument un dictionnaire `mdp` stockant les mots de passe des utilisateurices, un nom d'`utilisateur` et une `chaine` de caractère. Elle renvoie `True` si `chaine` est le mot de passe d'`utilisateur` stocké dans `mdp`, `False` sinon.

Code python
```python
def verifie(mdp, utilisateur, chaine):
    """ {str: str}, str, str -> bool
    Renvoie True lorsque chaine est le mot de passe associé à
    ↪ utilisateur dans mdp, False sinon """
    pass
```

Code python
```python
print(verifie(mdp, "Alice", "MVTMJSUNP"))
print(verifie(mdp, "Bob", "456"))
```

Résultat
```
True
False
```

8. **a.** Exécuter les codes suivants.

Code python
```python
for u in mdp:
    print(u)
```

Code python
```python
for u in mdp.keys():
    print(u)
```

Code python
```python
for u in mdp:
    print(u, mdp[u])
```

Code python
```python
for u, m in mdp.items():
    print(u, m)
```

   b. Lorsque `d` est dictionnaire, expliquer ce que réalise l'instruction `for k in d:`

2 Les résultats d'un référendum sont stockés dans une liste :

Code python
```python
votes = ["oui", "non", "oui", "oui", "oui", "non"]
```

1. La fonction `compte_voix` prend en argument une liste `votes` ne contenant que des `"oui"` et des `"non"`, et renvoie un dictionnaire :
   - dont les clés sont `"oui"` et `"non"`
   - la valeur associée à la clé `k` est le nombre de fois où `k` apparaît dans la liste `vote`

Code python
```python
def compte_voix(votes):
    """ [int] -> {str:int}
    Compte le nombre d'occurrences des votes 'oui' et 'non'. """
    res = {'oui': 0, 'non': 0}
    # À compléter
```

Code python
```python
print(compte_voix(votes))
```

Résultat
```
{'oui': 4, 'non': 2}
```

2. Écrire une fonction `resultat` qui renvoie la chaîne de caractères suivante :

   `Le {avis} l'emporte avec {pourcentage}% de votes !`

   où `{avis}` est l'avis majoritaire (`"oui"` ou `"non"`) et `{pourcentage}` est le pourcentage de votes ayant exprimé cet avis. En cas d'égalité, votre fonction renverra la chaîne de caractères `"Les deux avis sont à égalité"`.

Code python
```python
print(resultat(votes))
```

Résultat
```
Le oui l'emporte avec 66.67% de votes !
```

3 On dispose d'un dictionnaire qui associe à chaque prénom le numéro de son mois de naissance.

Code python
```python
naissances = {"Alvynne" : 3, "Boubakar": 8, "Clément": 1, "Émilie":8}
```

1. Que renvoie l'intruction `naissance["Alvynne"]` ?
2. Écrire une fonction `anniversaire` qui prend en argument un dictionnaire `naissances` et un `prenom` et qui renvoie numéro du mois d'anniversaire de `prenom`.
3. On souhaite écrire une fonction `anniversaires_du_mois` qui prend en argument le numéro d'un `mois` et qui renvoie la liste des prénoms des personnes dont l'anniversaire ce `mois`-ci.
   a. Quelles sont les valeurs successives prises par `p` lorsque l'on exécute l'intruction :

   `for p in naissances:`

   b. Compléter le code ci-dessous. On utilisera l'instruction `lst.append(elt)` pour **ajouter** l'élément `elt` à la **fin** de la liste `lst`.

Code python
```python
def anniversaires_du_mois(naissances, mois):
    """ {str: int}, int -> [str]
    Renvoie la liste des prénoms des personnes nées pendant le
    ↪ mois """
    prenoms = []
    for p in naissances:
        if ...:
            ....append(...)
    return ...
```

4 Le code Morse international, est un code permettant de transmettre un texte à l'aide de séries d'impulsions courtes et longues.
Dans ce code, on représente la lettre `'A'` par `'.-'` : pour transmettre la lettre `'A'` on utilise une impulsion courte `'.'`, puis une impulsion longue `'-'`.

Code python

```python
MORSE = {
    'A': '.-', 'B': '-...', 'C': '-.-.', 'D': '-..', 'E': '.',
    'F': '..-.', 'G': '--.', 'H': '....', 'I': '..', 'J': '.---',
    'K': '-.-', 'L': '.-..', 'M': '--', 'N': '-.', 'O': '---',
    'P': '.--.', 'Q': '--.-', 'R': '.-.', 'S': '...', 'T': '-',
    'U': '..-', 'V': '...-', 'W': '.--', 'X': '-..-', 'Y': '-.--',
    'Z': '--..',
    '0': '-----', '1': '.----', '2': '..---', '3': '...--',
    '4': '....-', '5': '.....', '6': '-....', '7': '--...',
    '8': '---..', '9': '----.',
    '.': '.-.-.-', ',': '--..--', '?': '..--..', "'": '.----.',
    '!': '-.-.--', '/': '-..-.', '(': '-.--.', ')': '-.--.-',
    '&': '.-...', ':': '---...', ';': '-.-.-.', '=': '-...-',
    '+': '.-.-.', '-': '-....-', '_': '..--.-', '"': '.-..-.',
    '$': '...-..-', '@': '.--.-.'
}
```

1. Répondre aux questions suivantes à l'aide des informations contenues dans le dictionnaire `code_morse`.

   a. Comment va-t-on envoyer le texte `'NSI'` ?

   b. On reçoit le message `'-...  .---  .-.'`. Pour faciliter le décodage, on a inséré le caractère `' '` entre les suites de `.` et `-` qui correspondent à une lettre.
   À quel texte cela-correspond-t-il ?

2. Écrire une fonction `encode` qui prend en argument un `message` écrit avec des caractères alphanumériques et qui renvoie la suite d'impulsions à envoyer en utilisant le code Morse.

   On supposera que tous les caractères de `message` correspondent à des clés du dictionnaire `MORSE`. Afin de faciliter la lecture du code morse résultant, on prendra soin de séparer les codes morse de chaque lettre par un espace.

Code python

```python
def encode(message):
    """ str -> str
    Renvoie le code morse correspondant au message """
```

Code python

```python
print(encode("SLTTS!"))
```

Résultat

```
... .-.. - - ... -.-.--
```

3. Dans cette question, on souhaite décoder une suite de `.` et de `-` correspondant à un message reçu en morse.

   a. Écrire une fonction `morse_decode` qui renvoie un dictionnaire dont :
   - les clés sont des séquences de `.` et de `-` ;
   - la valeur associé à la clé `k` est le caractère alphanunérique correspondant.

Code python

```python
def morse_decode():
    """ () -> {str: str}
    Renvoie le dictonnaire de décodage du code morse. """
```

Code python

```python
MORSE_DEC = morse_decode()
print(MORSE_DEC['...'])
print(MORSE_DEC['.-..'])
print(MORSE_DEC['-'])
```

Résultat

```
S
L
T
```

b. La fonction `decode` prend en argument une chaine de caractères `bips` constituée uniquement de `.` et de `-` correspondant à des codes morses et renvoie le message correspondant. Pour cela :

- pour chaque caractère `b` de `bips` :
  - s'il s'agit d'un `.` ou d'un `-` : alors on est en train de lire une séquence morse correspondant à un caractère. On ajoute ce caractère à la variable `code`.
  - s'il s'agit du caractère `' '` : alors on a finit de lire une séquence morse correspondant à un caractère. On utilise le dictionnaire `MORSE_DEC` pour obtenir le caractère alphanumérique correspondant à `code`, et on l'ajoute au `message`.

Code python

```python
def decode(bips):
    """ str -> str """
    message = ""
    code = ''
    # À compléter
```

Code python

```python
print(decode("... .-.. - - ... -.-.-- "))
```

Résultat

```
SLTTS!
```

5 Un refuge pour animaux dispose de plusieurs enclos. Il stocke ces informations dans une liste de dictionnaires.

Code python

```python
animaux = [
  {'nom': 'Medor', 'espece': 'chien', 'age': 5, 'enclos': 5},
  {'nom': 'Tom', 'espece': 'chat', 'age': 7, 'enclos': 4},
  {'nom': 'Belle', 'espece': 'chien', 'age': 6, 'enclos': 3},
  {'nom': 'Mirza', 'espece': 'chat', 'age': 6, 'enclos': 5}
]
```

1. Quelle instruction python permet de renvoyer le numéro de l'enclos de Médor ?
2. Écrire une fonction `animaux_refuge` qui prend en argument une liste d'`animaux` et renvoie la liste des noms de tous les animaux du refuge, tout enclos confondus.

Résultat

```python
assert animaux_refuge(animaux) == ['Medor', 'Tom', 'Belle', 'Mirza']
```

3. Écrire une fonction `animaux_enclos` qui prend en argument une liste d'`animaux` et un `numero` d'enclos et renvoie la liste des noms de tous les animaux présents dans l'enclos `numero`.

Résultat

```python
assert animaux_enclos(animaux, 5) == ['Medor', 'Mirza']
```

4. Écrire une fonction `informations` qui prend en argument une liste d'`animaux`, un `nom` d'animal, et qui renvoie le dictionnaire contenant toutes les informations relatives à `nom`.

Résultat

```python
assert informations(animaux, 'Medor') == {'nom': 'Medor', 'espece':
↪  'chien', 'age': 5, 'enclos': 5}
```